Homework 1 in CS646 Generative AI and Applications

Wang Po-Jen

March 22, 2026

1 Running the Baseline GAN

1.1 Dataset: Fashion MNIST

The model is trained on the **Fashion MNIST** dataset, which serves as a more complex benchmark than the original handwritten digits (MNIST).

- **Content:** A collection of 28×28 grayscale images of 10 fashion categories (e.g. bags, sneakers, trousers).
- **Data Split:** 60,000 training images are used to feed the Discriminator.
- **Preprocessing:**
  - **Reshaping:** Data are cast in `float32` and reshaped in $(28, 28, 1)$.
  - **Normalization:** Pixels are scaled from $[0, 255]$ to the range $[-1, 1]$. This is critical because the Generator uses a `tanh` activation function.
  - **Batching:** Data are shuffled and divided into mini-batches of 128 to stabilize gradient updates.

1.2 Model Architecture

The system utilizes two deep neural networks designed with Fully Connected (Dense) layers rather than Convolutional layers.

1.2.1 Generator Structure

The Generator transforms a 100-dimensional noise vector into a 28×28 image:

- **Hidden Layers:** Four Dense layers with increasing units ($128 \to 256 \to 512 \to 1024$).
- **Regularization:** `Batch` Normalization is applied after each dense layer (except the last) to prevent internal covariate shift.
- **Activation:** `LeakyReLU` (0.2) is used for hidden layers, while the output layer uses `tanh` to produce values in the range $[-1, 1]$.

1.2.2 Discriminator Structure

The Discriminator is a binary classifier that processes images:

- **Input Processing:** The 28×28 image is flattened into a 784-unit vector.
- **Hidden Layers:** Two Dense layers (512 and 256 units) using `LeakyReLU` activations.
- **Output Layer:** A single node with a `sigmoid` activation function, providing a probability score $D(x) \in [0, 1]$.

1.3 Loss Functions and Optimization

The implementation utilizes **Binary Cross-Entropy (BCE)** loss to facilitate the adversarial game.

1.3.1 Mathematical Formulation

The total objective is to solve the minimax problem:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))] \tag{1}$$

1.3.2 Implementation Details

- **Discriminator Loss:** Sum of the loss for real images (comparing $D(real)$ with a vector of ones) and fake images (comparing $D(fake)$ with a vector of zeros).
- **Generator Loss:** Calculated by comparing $D(fake)$ to a vector of ones, representing the Generator's goal to be perceived as "real."
- **Optimizers:** Two separate **Adam** optimizers are used with:
  - *Learning Rate:* 0.0002
  - *Beta coefficients:* $\beta_1 = 0.5, \beta_2 = 0.999$ (Low β_1 helps reduce training oscillations).

Figure 1: Generated image using the original GAN code.

2 Model Understanding

2.1 Explanation of Generator and Discriminator

The Generative Adversarial Network (GAN) framework is based on a game-theoretic scenario where two networks, the Generator (G) and the Discriminator (D), are trained simultaneously through an adversarial process.

2.1.1 The Generator (G)

The role of the Generator is to create realistic data. In the context of your code:

- **Function:** It takes a random noise vector z (sampled from a latent space, usually a Gaussian distribution) and maps it to the data space (e.g., a 28×28 image).
- **Objective:** To "fool" the Discriminator into classifying its fake outputs as real.
- **Analogy:** Think of the Generator as an art forger trying to create a masterpiece that looks like an original.

2.1.2 The Discriminator (D)

The Discriminator acts as a binary classifier.

- **Function:** It receives an input image x (either a real image from the Fashion MNIST dataset or a fake one from G) and outputs a probability score between 0 and 1.
- **Objective:** To correctly distinguish between "real" data (label 1) and "fake" data (label 0).
- **Analogy:** Think of the Discriminator as an art critic or authenticator who must decide if a painting is a genuine original or a forgery.

2.2 Instability in GAN Training

Training GANs is notoriously difficult because it involves finding a **Nash Equilibrium** in a high-dimensional, non-convex game, rather than simply minimizing a single objective function. Common instabilities include:

2.2.1 Mode Collapse

This occurs when the Generator discovers a small number of "safe" outputs that the Discriminator currently finds believable. Instead of learning the entire distribution of Fashion MNIST (e.g., shirts, shoes, and bags), the Generator might only produce many variations of a single "sneaker," failing to represent the diversity of the data.

2.2.2 Vanishing Gradients

If the Discriminator becomes too successful too early, it will classify all fake images as 0 with absolute certainty. This results in the gradient of the loss function becoming nearly zero, providing no useful information for the Generator to update its weights and improve.

2.2.3 Oscillation and Non-Convergence

Because the two models are constantly reacting to each other's updates, the training can enter a loop where the models oscillate between different states without ever reaching a stable point where the Generator produces high-quality images and the Discriminator is at 50% accuracy (random guessing).

2.2.4 Hyperparameter Sensitivity

GANs are highly sensitive to the choice of:

- **Learning Rates:** If one network learns significantly faster than the other, the adversarial balance is broken.
- **Optimizers:** Your code uses Adam with $\beta_1 = 0.5$, which is a specific heuristic to reduce momentum-driven oscillations in GANs.

3 Model Improvements

3.1 Architectural Shift: From MLP to CNN

The most significant improvement is the transition of the Discriminator from a Multi-Layer Perceptron (MLP) to a Convolutional Neural Network (CNN).

- **Before:** The model used `layers.Dense` units which flattened the 28×28 image into a 784-dimensional vector. This approach treats pixels as independent variables, completely ignoring the spatial structure and local correlations.
- **After:** It utilizes `layers.Conv2D` with strides. Convolutional layers act as feature extractors that can recognize patterns such as edges, textures, and shapes. This raises the "intelligence ceiling" of the Discriminator, allowing it to provide more sophisticated gradients to the Generator.

3.2 Introduction of Dropout for Regularization

The second version introduces `layers.Dropout(0.3)` after each LeakyReLU activation in the Discriminator.

- **Purpose:** In GAN training, a common failure mode is the Discriminator becoming "too strong" too quickly, leading to vanishing gradients for the Generator.
- **Improvement:** Dropout introduces noise by randomly disabling neurons during training. This prevents the Discriminator from over-relying on a few specific pixels to "cheat" and forces it to learn more robust, diverse features. This is a key defense against Mode Collapse.

3.3 Heuristic Asymmetric Learning Rates

The optimization strategy was adjusted from symmetric to asymmetric learning rates to balance the adversarial game.

- **Before:** Both networks used a uniform learning rate of 0.0002.
- **After:** The Discriminator learning rate is set to 0.0008, while the Generator learning rate is 0.0001 (an 8:1 ratio).
- **Improvement:** This specific ratio addresses the "learning lag" observed in previous loss plots. By allowing the Discriminator to learn faster, it maintains constant pressure on the Generator, preventing it from stagnating at a low-quality output.

3.4 One-Sided Label Smoothing

The loss calculation for real images was modified to include label smoothing techniques.

- **Change:** Using `tf.ones_like(real_output) * 0.9` instead of a hard target of 1.0.
- **Improvement:** Instead of targeting a hard "1.0" for real images, the Discriminator targets "0.9". This prevents the Discriminator from becoming "overconfident," which often leads to extreme gradient values that can destabilize the Generator's weights. Smoothens the optimization landscape, making the training process more resilient.

3.5 Enhanced Monitoring and Visualization

The training loop was upgraded to provide better feedback on the model's convergence and internal state.

- **Added Metrics:** The code now tracks `G_history`, `D_history`, and Discriminator Scores (Real vs. Fake).
- **Improvement:** Viewing the loss alone is often insufficient for GANs. By monitoring the scores (the mean output of the Discriminator), one can determine if the two networks are reaching a Nash Equilibrium. When both scores hover near 0.5, indicates that the Generator is successfully producing images that the Discriminator can no longer easily distinguish from reality.

4 Visualization & Results

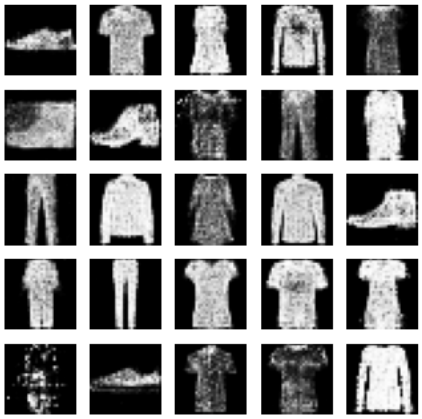
4.1 Generative Adversarial Network (GAN) Losses

The training process is a zero-sum game between the Generator (G) and the Discriminator (D), which can be formally described and optimized through the following components:

- **Minimax Objective Function:**The overall goal is to find the Nash Equilibrium of the value function $V(D, G)$:

$$\min_G \max_D V(D, G) = \mathbb{E}x \sim pdata[\log D(x)] + \mathbb{E}z \sim pz[\log(1 - D(G(z)))] \tag{2}$$

- **Discriminator Loss (L_D):** The Discriminator is trained as a binary classifier to maximize the accuracy of its labels.
  - **Real Loss:** Calculated as $-\log D(x)$, which is minimized when $D(x)$ approaches 1.
  - **Fake Loss:** Calculated as $-\log(1-D(G(z)))$, which is minimized when $D(G(z))$ approaches 0.
- **Generator Loss (L_G):** The Generator aims to produce samples that the Discriminator classifies as real data.
  - **Non-Saturating Loss:** Instead of minimizing the original minimax term, G typically minimizes $-\log D(G(z))$. This provides stronger gradients in the early stages of training when the Generator is weak.
- **One-Sided Label Smoothing:** A regularization technique used to prevent the Discriminator from becoming too dominant.
  - **Mechanism:** The target label for real images is reduced from 1.0 to 0.9.
  - **Purpose:** This prevents the Discriminator from producing extremely large gradients that could destabilize the Generator's updates and helps mitigate the vanishing gradient problem.

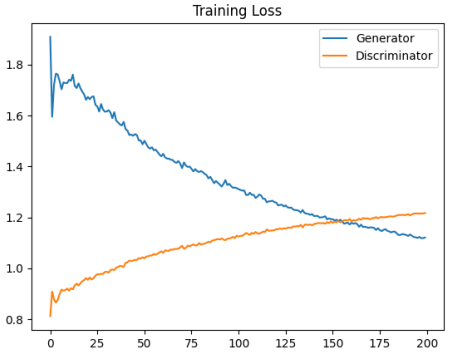


(a) Before

(b) After

Figure 2: Comparison of before/after results.

(a) Losses before modified



(b) Losses after modified

Figure 3: Comparison of before/after losses.

5 Comparative Analysis

5.1 Architectural Evolution

The most critical change is the shift in how the Discriminator perceives data.

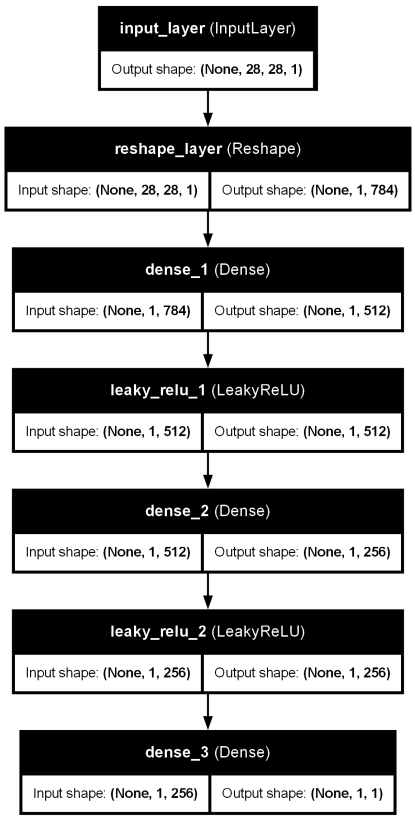
- **Original (MLP-based):** Used `layers.Dense` units. It flattens the 28×28 image into a 1D vector of 784 pixels. This ignores the spatial relationship between neighboring pixels, making it harder to learn complex patterns such as "textures" or "shapes."
- **Modified (CNN-based):** Uses `layers.Conv2D`. Convolutional filters slide across the image to detect local features. This allows the Discriminator to understand the "geometry" of clothing items (e.g., the curve of a shoe or the symmetry of a shirt).

5.2 Strategic Improvements

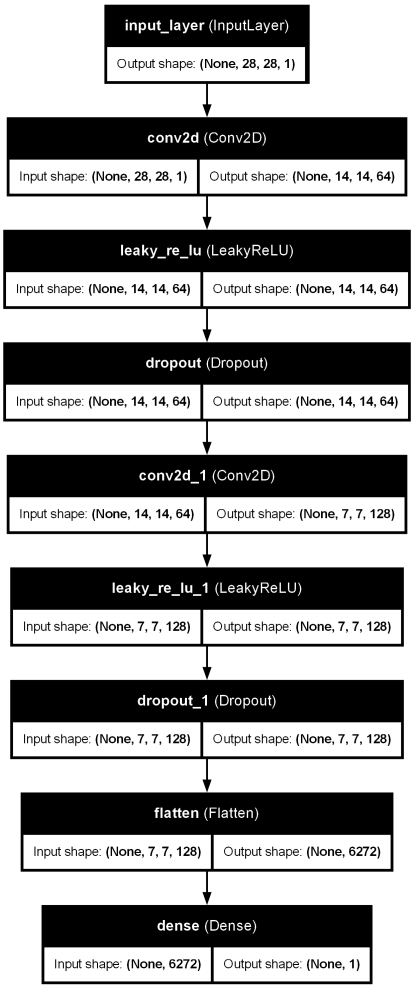
Beyond the layers themselves, the training dynamics was tuned to prevent common GAN failure modes.

- **Regularization (Dropout):** The modified version adds `Dropout(0.3)` to the Discriminator. This prevents the Discriminator from becoming "too powerful" too quickly, which usually causes the Generator's gradients to vanish.
- **Asymmetric Learning Rates:**
  - **Original:** Both used 0.0002.
  - **Modified:** Discriminator (0.0008) vs. Generator (0.0001).
  - **Result:** A faster Discriminator provides a more stable "moving target" for the slower, more deliberate Generator to follow.
- **Label Smoothing:** Replace the hard target 1.0 with 0.9 for real images. This "softens" the loss, preventing the Discriminator from producing extreme mathematical values that could destabilize the entire network.

5.3 Summary Comparison Table



(a) Original discriminator model



(b) Modified discriminator model

Figure 4: Comparison of before/after Tables.

5.4 Impact on Results

While the Original version might generate blurry, blob-like shapes that vaguely resemble clothes, the Modified version is much more likely to produce sharp edges and distinct fashion items. The addition of history tracking also allows you to see if the model is reaching a Nash Equilibrium, where neither network can significantly improve at the expense of the other.

6 GitHub Submission

`https://github.com/wangboren/Homework-1-in-CS646-Generative-AI-and-Applications`

References

Source Code: `https://colab.research.google.com/github/qaazii/3D-Bounding-Boxes-From-Monocular-Images/blob/master/Intro-to-Generative-Adversarial-Network/Tensorflow/GAN_Tensorflow_Fashion_MNIST.ipynb`